

GeoMelody: a Context-aware Music Recommendation System

Alice Zhang
az536@cornell.edu
Cornell Tech
USA

Jessica Chi
hc2287@cornell.edu
Cornell Tech
USA

Dingran Dai
dd699@cornell.edu
Cornell Tech
USA

Thalia Zhang
yz3474@cornell.edu
Cornell Tech
USA

Abstract

GeoMelody is a context-aware music recommendation system that grounds song recommendations in the listener’s real-world environment and physiological state at the moment of listening. The system integrates a CircuitPython-based wearable device, a FastAPI backend, and a Next.js web application to generate personalized music recommendations from the user’s Spotify library. The wearable continuously streams contextual signals, including heart rate, ambient noise, and motion state, to the backend, while the web application combines these signals with user-selected contextual information such as scene and mood. Based on these inputs, the system computes a five-dimensional target audio profile and uses Gemini 2.5 Flash to recommend tracks that align with the user’s current experience. Beyond music recommendation, GeoMelody includes a reflective feature called “Memory Cards,” which allows users to preserve listening sessions through generated poems or word clouds derived from recommended tracks and associated sensor data. This paper presents the system’s design rationale, architecture, and core features, as well as challenges encountered during development, including a mid-project pivot prompted by Spotify’s deprecation of its audio-features endpoint. Finally, the paper discusses limitations of the current prototype and directions for future work. The source code for the project is publicly available at: <https://github.com/thaliazhang03/geomelody>.

ACM Reference Format:

Alice Zhang, Dingran Dai, Jessica Chi, and Thalia Zhang. 2026. GeoMelody: a Context-aware Music Recommendation System. In . ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

When people travel or explore new places, the music recommended by existing streaming platforms often does not match the user’s surroundings or current activity. A visitor to Miami, for example, may receive the same high-energy party playlist regardless of whether they are dancing in a club or watching the sunset on the beach.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Walking through a quiet space such as the Vizcaya Garden Museum, by contrast, may call for music that is calmer or more atmospheric, such as classical or instrumental tracks. In these situations, recommendations driven primarily by listening history can feel disconnected from the listener’s immediate environment, interrupting immersion and undermining the overall listening experience.

GeoMelody is a context-aware music recommendation system designed to address this limitation. Instead of relying solely on historical listening patterns, the system generates recommendations based on the user’s real-time physiological and environmental context. A wearable device streams heart rate, ambient noise, and motion data to a backend server, while a web application combines these signals with user-selected contextual inputs such as scene and mood. The system then computes a target audio profile and uses a large language model to select five tracks from the user’s Spotify library that best align with the current context. By grounding recommendations in the listener’s present experience while still drawing from familiar music preferences, GeoMelody aims to create a more immersive and contextually appropriate listening experience.

The contributions of this work are threefold. First, we present a complete sensor-to-music pipeline that integrates physiological, environmental, and motion signals with user-provided mood information to better distinguish between physiologically similar states, such as stress and excitement. Second, we describe a hybrid recommendation framework in which a rule-based system computes a five-dimensional target audio profile and a large language model performs the final music selection, serving as a robust alternative following Spotify’s deprecation of its audio-features endpoint. Third, we introduce an interactive web application prototype that simulates the interaction flow of existing music streaming platforms, including functionalities such as music selection, playback control, and queue management.

2 Related Work

Prior research has explored both contextual and physiological dimensions of music recommendation. One line of work proposes matching music to points of interest (POIs) using emotional tags, showing that contextual information about location can help identify music appropriate to a particular place or situation [1]. This validates the intuition that the surrounding environment is a meaningful signal for music selection, distinct from long-term taste profiles.

A second line of work investigates physiological sensing, using signals such as heart rate and galvanic skin response to infer emotional state and adjust playlists accordingly [2, 3]. These systems demonstrate that physiological data can be used not only to mirror the listener’s current arousal, but also to actively guide them toward a more stable emotional state through music selection.

GeoMelody builds upon both lines of prior work. Similar to POI-driven recommendation systems, it treats environmental context as a primary input to the recommendation process rather than relying solely on historical listening behavior. At the same time, like physiological music recommendation systems, it incorporates real-time bodily signals to capture aspects of the listener’s current state that may not be reflected through explicit interaction alone.

However, GeoMelody differs from existing approaches in two key ways. First, the system incorporates an explicit user-provided activity and mood anchor (e.g., Focused, Relaxed, Stressed, or Energetic) at the beginning of each session to help disambiguate physiological signals that may otherwise correspond to multiple emotional or situational interpretations. For example, an elevated heart rate may reflect either stress or excitement depending on the surrounding context. Second, rather than recommending music from a global catalog, GeoMelody limits recommendations to tracks already present in the user’s Spotify library. This design choice allows the system to adapt recommendations to the listener’s current context while remaining grounded in the user’s existing music preferences.

3 System Overview

GeoMelody is a context-aware music recommendation system that maps real-world sensor data, including heart rate, ambient noise level, and movement state, into personalized Spotify track suggestions. Rather than relying on listening history alone, GeoMelody grounds recommendations in what the user is physically experiencing at the moment they press play.

The system integrates three hardware and software layers: a CircuitPython wearable that streams biometric and environmental readings over Wi-Fi, a FastAPI backend that stores and serves those readings, and a Next.js frontend that authenticates with Spotify, pulls the user’s track library, and queries the Gemini API to select and explain the five best matches for the current context. Table 1 summarizes the responsibilities of each layer.

Table 1: Component responsibilities.

Component	Technology	Role
Wearable	CircuitPython	Streams sensor payload.
Backend	FastAPI (Python)	Caches and serves data.
Frontend	Next.js 16	OAuth, UI, model call.
Recommender	Gemini 2.5 Flash	Picks 5 tracks + reasons.

Data flows in one direction: the wearable POSTs to /sensor-data on the backend, the frontend pulls from /latest-sensor-data, and the frontend then issues a structured request to the Gemini API. Tracks are pre-fetched from the user’s Spotify library at session start, so the model only needs to choose among songs the user already owns rather than retrieve them.

3.1 Hardware

The wearable is built around an Adafruit QT ESP32-S3 microcontroller running CircuitPython. It carries three sensors: a pulse-oximetry module for heart rate, an electret microphone for ambient noise, and a six-axis IMU (LSM6DS3) for motion classification. The board connects over Wi-Fi and samples each sensor at a high rate locally, then transmits an averaged reading to the backend approximately every three minutes. Each signal is pre-processed before transmission:

- **Heart rate.** A DC-removal filter strips the slow-moving baseline from the IR filter, after which a smoothing filter is applied to suppress high-frequency noise from minute skin friction. Beat detection is done in the time domain by calculating the time difference between pulses when the signal drops below a threshold percentage of its recent amplitude range. BPM is then calculated using the latest four consecutive beat intervals.
- **Ambient noise.** The microphone signal is converted to a root-mean-square amplitude and then to decibels (dB), giving a perceptual loudness estimate.
- **Activity state.** The IMU runs a rolling-window classifier over the last 100 samples and emits one of three labels based on signal variance: ACT_STILL, ACT_WALKING, or ACT_WORKING.

Streaming an averaged reading rather than every sample keeps power and bandwidth usage low while still capturing meaningful shifts in the user’s state between recommendation refreshes. During development and testing we shortened the push interval to roughly five seconds to verify end-to-end latency; the deployed configuration uses the three-minute averaging window.

3.2 Backend

The backend consists of a FastAPI server that serves as the communication layer between the wearable device and the web application. The server exposes two endpoints. The POST /sensor-data endpoint receives heart rate, ambient noise, and IMU readings streamed from the wearable device and stores the latest values in an in-memory cache. The GET /latest-sensor-data endpoint allows the frontend to retrieve the most recent sensor readings for recommendation generation. Since the system was designed as a single-device prototype, persistent storage and per-user session management were intentionally omitted in order to reduce implementation complexity and deployment overhead. CORS was configured to allow communication between the frontend and backend during development.

While the FastAPI server primarily functions as a lightweight sensor relay, the main recommendation pipeline is implemented within the backend logic of the Next.js application. The recommendation process begins by retrieving the latest sensor readings and normalizing the raw physiological and environmental values. These normalized signals are then combined with user-selected contextual inputs, including scene, mood, and activity, to generate a five-dimensional target audio profile. Finally, the generated profile, along with a sampled set of seed tracks from the user’s Spotify library, is passed to an LLM for track selection.

3.2.1 Sensor Normalization. Before any sensor value enters the recommendation engine, it is normalized to the closed interval $[0.0, 1.0]$. This decouples the engine from the firmware’s raw measurement scale and makes the engine’s behavior easier to reason about.

For ambient noise, the linear mapping

$$\text{ENV} = \frac{dB - 30}{60}$$

places a quiet environment (~ 30 dB) at 0.0 and a noisy one (~ 90 dB) near 1.0. As a worked example, $dB = 60$ yields $\text{ENV} = 0.50$.

For heart rate, normalization is anchored to the user’s own resting baseline measured during the first 30 seconds of a session:

$$S_{hr} = \frac{HR_{\text{current}} - HR_{\text{baseline}}}{HR_{\text{max}} - HR_{\text{baseline}}}$$

with $HR_{\text{max}} = 200$. A user whose baseline is 70 and whose current heart rate is 90 yields $S_{hr} = 20/130 \approx 0.15$. Per-user calibration is essential here: a resting rate of 60 and a resting rate of 80 produce very different absolute readings for the same level of arousal, so anchoring S_{hr} to the user’s own baseline rather than a population average preserves comparability across listeners.

For the IMU, the three discrete labels are mapped to fixed scalars: $\text{ACT_STILL} = 0.0$, $\text{ACT_WALKING} = 0.5$, and $\text{ACT_WORKING} = 1.0$.

3.2.2 Target Audio Profile. The recommendation engine represents the desired musical characteristics of a listening session using a five-dimensional target audio vector consisting of energy, danceability, valence, instrumentalness, and acousticness. The vector is initialized using a scene-dependent baseline profile (Table 2) corresponding to the user’s selected environment. It is then adjusted using additional modifiers derived from the user’s selected mood and real-time sensor signals, including heart rate, ambient noise, and motion data. After these adjustments are applied, each dimension of the vector is clamped to the range $[0.0, 1.0]$ to ensure valid audio feature values.

Each dimension of the target audio profile is subsequently adjusted according to its role in the listening experience. Energy and danceability are influenced primarily by physiological arousal and motion state, while valence and instrumentalness are modified only for specific mood conditions, namely *Stressed* and *Focused*, respectively. Acousticness is adjusted solely based on ambient noise levels. The noise mapping is implemented as a linear relationship in which louder environments shift recommendations toward more electronic and higher-energy music by decreasing acousticness by $S_{\text{noise}} \times 0.40$ and increasing energy by $S_{\text{noise}} \times 0.15$. Conversely, quieter environments bias the recommendation profile toward more acoustic textures. The rationale behind this design choice is that environmental conditions constrain which types of music remain perceptually effective in practice. For example, quieter acoustic tracks may be difficult to perceive clearly in loud environments such as subway platforms regardless of the listener’s preferences.

3.2.3 Mood and Heart Rate. Heart rate alone is ambiguous: an elevated reading can indicate either stress or excitement, and the system cannot resolve that ambiguity from the sensor signal alone. Mood, selected by the user once at the start of a session, supplies the missing context. Combined with a threshold on S_{hr} (≥ 0.65 is

treated as “high”), it determines how the engine shifts the target vector. The offsets are listed in Table 3.

The intent behind these offsets is straightforward. A *Focused* listener with a high heart rate is likely working under pressure, so the engine lowers energy and increases instrumentalness to reduce lyrical distraction. A *Stressed* listener receives a stronger downward adjustment to energy together with an upward adjustment to valence, since the goal is to guide the listener toward a calmer state rather than simply mirror their current arousal. By contrast, an *Energetic* listener with a high heart rate is assumed to be positively engaged in their activity, so the engine amplifies energy and danceability rather than suppressing them.

3.2.4 Activity Offset (IMU). Once the mood offsets have been applied, a final adjustment is layered on top based on the activity classification reported by the IMU. This step is independent of mood: being still lowers danceability regardless of how the listener feels about it. Walking adds energy and danceability; sustained work calls for more instrumental texture and slightly less energy. The full table of activity offsets is given in Table 4.

3.2.5 Track Selection via LLM. Once the target vector is constructed, the frontend calls the Gemini API with a structured prompt containing the sampled seed list (twenty-five tracks drawn uniformly at random from the user’s library, with previously shown tracks filtered out), the computed target vector, and the detected scene, mood, and activity labels. The model is asked to return exactly five tracks from the seed list, each with a one-sentence rationale of at most eight words. The response is constrained by a JSON schema so that the frontend can parse it deterministically.

Our original design relied on Spotify’s audio-features endpoint to retrieve track-level audio features and rank candidate songs using Euclidean distance to the target audio vector. However, Spotify recently deprecated this endpoint for new applications, requiring a redesign of the recommendation pipeline midway through development. As an alternative, we initially explored a keyword-overlap approach based on artist-level genre labels, but this method proved overly coarse-grained because all tracks by the same artist shared identical genre descriptors regardless of differences in musical style or mood. For example, tracks from the same jazz artist could receive identical genre representations despite differing substantially in tempo, instrumentation, or emotional tone.

To address this limitation, we retained the rule-based target vector computation while delegating the final recommendation selection process to Gemini. This approach preserved much of the contextual matching capability originally enabled by Spotify’s audio features while avoiding the brittleness of keyword-based matching. The original feature-vector implementation (ruleEngine.ts, euclidean.ts) is included in the repository as documentation of this design pivot.

3.3 Frontend

The frontend is implemented as a client-rendered Next.js 14 application. User authentication is handled through Spotify’s PKCE OAuth flow, which eliminates the need for storing a client secret and allows the authentication process to occur entirely within the

Table 2: Scene baselines for the five-dimensional target audio profile.

Scene	Energy	Danceability	Acousticness	Valence	Instrumentalness
Library	0.30	0.20	0.50	0.50	0.85
Café	0.45	0.40	0.60	0.70	0.30
Street	0.70	0.75	0.20	0.60	0.20
Subway	0.85	0.60	0.05	0.40	0.20
Park	0.55	0.50	0.80	0.85	0.60

Table 3: Mood and heart-rate offsets, added on top of the scene baseline.

Mood (and S_{hr})	Energy	Danceability	Valence	Instrumentalness
Focused ($S_{hr} \geq 0.65$)	-0.15	-	-	+0.20
Focused ($S_{hr} < 0.65$)	-0.05	-	-	+0.10
Relaxed	-0.20	-0.15	-	-
Stressed ($S_{hr} \geq 0.65$)	-0.25	-	+0.20	-
Stressed ($S_{hr} < 0.65$)	-0.10	-	+0.10	-
Energetic ($S_{hr} \geq 0.65$)	+0.20	+0.15	-	-
Energetic ($S_{hr} < 0.65$)	+0.08	+0.08	-	-

Table 4: IMU activity offsets, applied after the mood offsets.

Activity	Energy	Danceability	Instrumentalness
ACT_STILL	-	-0.10	-
ACT_WALKING	+0.10	+0.15	-
ACT_WORKING	-0.05	-	+0.10

browser. The resulting access token is stored in `sessionStorage` and removed upon logout.

The primary interaction flow is organized across six stages. Users first connect their Spotify account, then choose a track source, either Liked Songs or Top Tracks from the previous six months. Users subsequently configure the listening context by selecting a scene and mood, while activity state is automatically inferred from IMU readings when the wearable device is connected. The system then presents five recommended tracks together with short recommendation rationales, and tracks can be previewed directly within the interface using Spotify’s Web Playback SDK. Users may optionally curate tracks into a “Memory Card” before browsing a personal gallery of previously saved cards. In addition, the interface includes an animated visualization called the DotOrb, which responds to microphone input for visual feedback only and does not influence recommendation generation.

We made two key frontend design decisions. First, sensor context is *frozen at curation time*. When a user saves a recommended track, the associated scene, mood, activity state, and normalized sensor readings are stored alongside it. As a result, contextual information remains tied to the moment in which the recommendation was generated, even if the user’s environment changes later in the session. This allows Memory Cards generated at a later time to accurately reflect the conditions under which tracks were originally curated. Second, the gallery is keyed by Spotify user ID rather than by a device-wide identifier. If the application fails to retrieve

the current Spotify user identity through the `/v1/me` endpoint, the system intentionally refuses to save the card under a placeholder identifier in order to avoid orphaned or inaccessible saved sessions. The trade-off is that saved galleries remain device-local in the current implementation, a limitation that could be addressed through future server-side storage.

3.3.1 Memory Cards. Listening sessions are often ephemeral, with contextual associations disappearing once playback ends. To address this issue, we designed a “Memory Cards” feature to preserve aspects of these listening moments by generating shareable artifacts derived from curated tracks and their associated contextual metadata. Once three or more tracks have been curated during a session, users can enter the Memory Card flow through the Share interface.

The system currently supports two output formats. The first, a **Poem Card**, uses Gemini to generate a short free-verse poem in which song titles are incorporated as imagery rather than directly quoted. The generated writing style is partially conditioned on contextual sensor readings: elevated heart rate values produce shorter and more fragmented lines, louder environments bias toward abrupt phrasing, while calmer contexts yield longer and more stable sentence structures. In addition, a forbidden-word list (*vibe, perfect, journey, embrace, fits, soul, magic, beautiful, ethereal, heart*) is used to suppress generic AI-generated praise language in favor of more concrete sensory descriptions.

The second format, a **Word Cloud Card**, visualizes curated song titles using an Archimedean spiral layout in which title size corresponds to relative frequency and emphasis. Song titles selected by Gemini as particularly representative of the session are rendered larger and highlighted using the palette accent color. Both Memory Card format share an eight-color palette derived from the aggregate scene and mood information of the curated tracks, allowing different card types generated from the same session to maintain a consistent visual identity.

4 Limitations and Future Work

In this section, we discuss limitations associated with the current implementation of each component of the system, as well as potential directions for future work.

Hardware. The wearable must be currently externally powered via the USB terminal. This is due to incompatibility with the QT ESP32-S3, which does not have a built-in Lipo battery terminal. Future iterations would explore integration of all sensors and Lipo compatibility onto a custom PCB board to minimize the hardware form factor and allow for better wearability. The device also currently pushes sensor data at a fixed cadence, which is inefficient when the user remains still for long periods and may under-sample during rapid transitions. A battery-aware push policy that lengthens the interval when the IMU reports stillness and shortens it during movement could extend future battery life. The pulse-oximetry module also produces noisy readings due to being sensitive to movement noise. Future signal processing solutions would be to explore frequency domain analysis for more robust BPM readings.

Backend. The sensor status card on the frontend currently displays “live” as long as the backend contains any cached reading, even if the device disconnected several minutes earlier. Adding a last_seen heartbeat and updating the UI accordingly when the device becomes inactive would improve the accuracy of the system state representation. The current in-memory cache also limits the system to a single active device and does not support persistent storage of listening sessions. Migrating to a database system keyed by device or user ID would enable history-based features such as revisiting previous listening sessions.

Frontend. Saved “Memory Cards” currently reside in browser localStorage and are therefore tied to a single device and browser session. A server-side gallery associated with Spotify user IDs would allow users to access their saved cards across devices. In addition, future iterations of the frontend may incorporate a visual context inference feature that allows users to upload or capture an image of their surroundings. A corresponding vision-based endpoint could then infer contextual attributes such as scene, activity, and mood from the image to supplement sensor-based inputs.

Recommendation quality. Skip behavior and play-through rates are not currently incorporated into the recommendation pipeline. Even coarse feedback signals, for example, whether a user skipped multiple recommended tracks, could improve subsequent recommendations by allowing the system to adapt over time. The current sampling strategy has also only been evaluated on libraries ranging from roughly fifty to five hundred tracks. Larger libraries may require more sophisticated retrieval and sampling approaches, such as genre-aware or recency-aware sampling, rather than the current uniform random selection strategy.

5 Conclusion

GeoMelody was designed around the idea that contextually appropriate music recommendations can often be generated from tracks the listener already knows and enjoys, rather than requiring discovery from a broader catalog. By combining real-time physiological and environmental signals, including heart rate, ambient noise, and motion state, with user-provided contextual inputs such as mood and scene, the system narrows a potentially large personal music

library to a small set of tracks that best fit the listener’s current situation. A large language model performs the final recommendation step, which ultimately proved to be a more robust alternative following Spotify’s deprecation of its audio-features endpoint than the keyword-based fallback strategy we initially explored.

Our broader takeaway from this project is the importance of minimizing user effort in the design of context-aware systems. Wherever possible, contextual information may be more effectively inferred automatically rather than requiring continuous manual input. For example, in GeoMelody, activity state is inferred from IMU readings instead of being explicitly selected by the user, while scene and mood are configured only once at the beginning of a listening session. Similarly, the process of preserving a session through “Memory Cards” is implemented as a lightweight interaction in which contextual metadata is captured automatically alongside the recommended tracks. Overall, GeoMelody is intended to function as a subtle companion to the listening experience rather than a system that continuously competes for the user’s attention.

6 AI Usage Statement

We used GPT and Claude during development for debugging code and refining the phrasing of the final report. Gemini was integrated into the GeoMelody system itself for generating song recommendations and producing text for the “Memory Card” feature.

References

- [1] Matthias Braunhofer, Marius Kaminskas, and Francesco Ricci. 2013. Location-aware music recommendation. *International Journal of Multimedia Information Retrieval* 2, 1 (2013), 31–44.
- [2] Joris H. Janssen, Egon L. van den Broek, and Joyce H. D. M. Westerink. 2012. Tune in to your emotions: a robust personalized affective music player. *User Modeling and User-Adapted Interaction* 22, 3 (2012), 255–279.
- [3] Hao Liu, Jun Hu, and Matthias Rauterberg. 2009. Music playlist recommendation based on user heartbeat and music preference. In *Proceedings of the International Conference on Computer Technology and Development (ICCTD)*. 545–549.

A Interface Screenshots

Example screenshots of the GeoMelody interface are shown below.

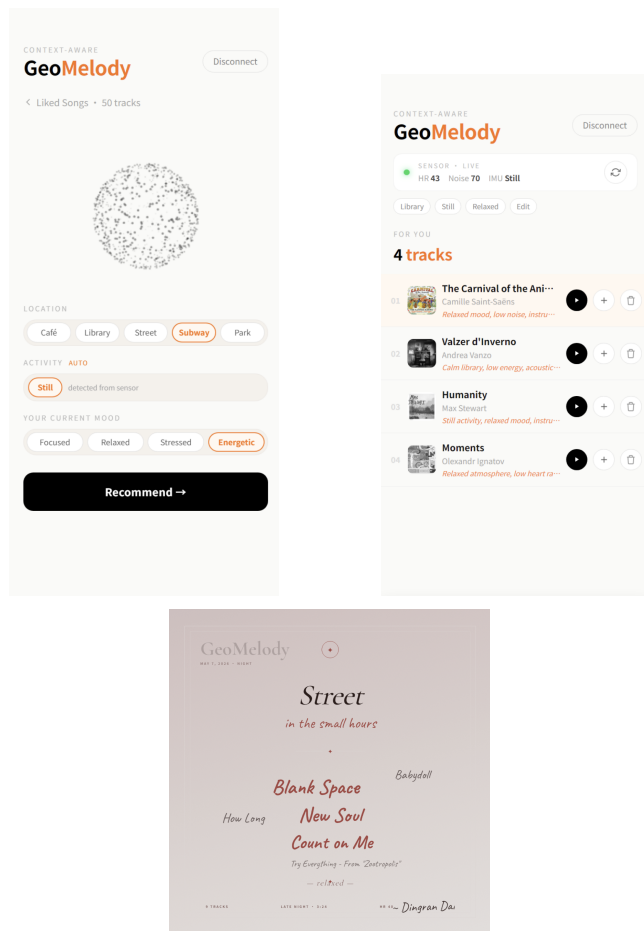


Figure 1: GeoMelody interface screenshots.

B Device Prototype

Images of the GeoMelody wearable device prototypes are shown below.



Figure 2: GeoMelody wearable device prototypes.

C Prompt Used for Recommendation Generation

```
You are the GeoMelody Engine a music recommendation
system.
Select exactly 5 tracks from the seed list that best
match the target audio profile.

== Sensor Context ==
Location : ${scene}
Activity : ${activity}
Mood : ${mood}
S_hr : ${sensor.heart_rate.toFixed(2)} (heart rate
arousal, 0.01.0)
S_noise : ${sensor.noise_level.toFixed(2)}
(environment noise, 0.01.0)

== Target Audio Profile (pre-computed) ==
energy : ${targets.energy.toFixed(4)}
danceability : ${targets.danceability.toFixed(4)}
acousticness : ${targets.acousticness.toFixed(4)}
valence : ${targets.valence.toFixed(4)}
instrumentalness : ${targets.instrumentalness.toFixed(4)}

== Seed List (${tracks.length} tracks) ==
${tracksJson}

== Instructions ==
1. Select 5 unique tracks from the seed list. Each one
must come from the list above do not invent tracks.
2. For each selection, return the EXACT "id" string from
the seed list (copy it verbatim do not modify,
shorten, or guess).
3. Choose tracks whose genre, style, and energy best fit
the target audio profile.
4. The "reason" field must be max 8 words. Be concrete
about how sensor music maps.
Good: "High noise; this track cuts through."
Better: "Loud subway; punchy bass holds focus."
Bad: "It fits the vibe."
5. Do not repeat ids. All 5 ids must be distinct.`
```